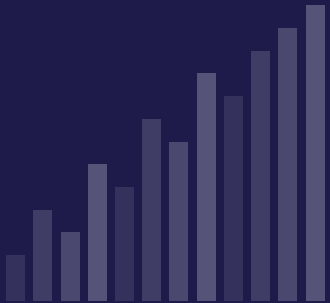
AULA 17 · BLOCO 2: ORDENAÇÃO AVANÇADA E ESTRUTURAS LINEARES

Listas ligadas e sentinelas

Cormen et al., *Algoritmos*: seções 10.2 e 10.3



TEORIA

Roteiro da aula

Ao final, você será capaz de

- Implementar busca, inserção e remoção em listas duplamente ligadas
- Usar uma sentinela para eliminar casos especiais
- Representar listas com vetores (sem ponteiros)
- Medir a vantagem da lista na inserção e o custo do cache no percurso

Teoria
Listas duplamente ligadas, sentinelas, listas em vetores

Prática em C
Lista com sentinela em C

Experimento
Inserir no início; percorrer uma lista espalhada pela memória

Conexões
Sistemas operacionais, editores, alocação de memória e cache



Teoria

- Cormen et al., seções 10.2 e 10.3
- Lista duplamente ligada
- Sentinelas
- Listas sem ponteiros

TEORIA

Lista duplamente ligada

Cada nó tem *chave*, *prox* e *ant*. LIST-SEARCH: $\Theta(n)$ no pior caso. LIST-INSERT na cabeça: $O(1)$. LIST-DELETE dado o nó: $O(1)$.

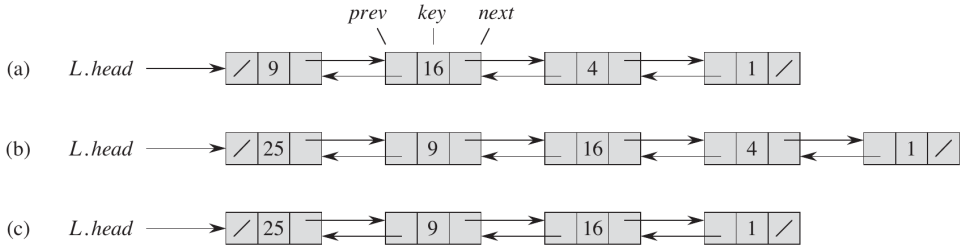


Figura 10.3 de Cormen et al. (2009) — (a) lista $\langle 9, 16, 4, 1 \rangle$; (b) após LIST-INSERT de 25; (c) após remover o 4.

TEORIA

Sentinelas

- Um nó falso $L.nil$ fica entre a cabeça e a cauda.
- $L.nil.prox$ é a cabeça; $L.nil.ant$ é a cauda.
- Remover passa a ser só: $x.ant.prox \leftarrow x.prox$; $x.prox.ant \leftarrow x.ant$.
- Sem testes de lista vazia ou de extremos: código menor e mais difícil de errar.

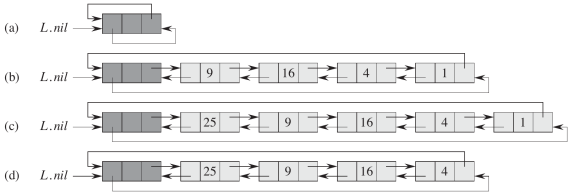


Figura 10.4 de Cormen et al. (2009) — lista circular com sentinela $L.nil$: (a) vazia; (b)–(d) inserções e remoções.

TEORIA

Listas sem ponteiros

- Linguagens sem ponteiros (ou dados em disco) usam índices como “ponteiros”.
- Os nós livres formam uma **lista livre**, usada como pilha: ALLOCATE-OBJECT e FREE-OBJECT em $O(1)$.
- É assim que alocadores de memória e sistemas de arquivos gerenciam blocos.

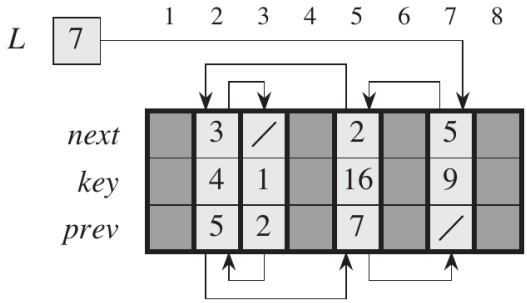


Figura 10.5 de Cormen et al. (2009) — a lista da Figura 10.3(a) guardada em três vetores *next*, *key* e *prev*.



Prática em C

- `struct No` com `ant` e `prox`; sentinela dentro da lista
- Inserção na cabeça, busca e remoção sem casos especiais
- Comandos: `i`, `b`, `r`, `p`

PRÁTICA EM C

Lista com sentinela em C (1/4)

```c
#include <stdio.h>
#include <stdlib.h>

/* A sentinela L->nil é um nó falso: nil->prox é a cabeça e nil->ant é a cauda.
   Com ela, inserir e remover não têm casos especiais (lista vazia, extremos). */
typedef struct No { int chave; struct No *ant, *prox; } No;
typedef struct { No nil; } Lista;

void inicia(Lista *L) { L->nil.prox = L->nil.ant = &L->nil; }
```

PRÁTICA EM C

Lista com sentinela em C (2/4)

```c
void insere(Lista *L, int k) {                  // LIST-INSERT': na cabeça, O(1)
    No *x = malloc(sizeof(No));
    x->chave = k;
    x->prox = L->nil.prox;
    L->nil.prox->ant = x;
    L->nil.prox = x;
    x->ant = &L->nil;
}

No *busca(Lista *L, int k) {                    // LIST-SEARCH': θ(n) no pior caso
    No *x = L->nil.prox;
    while (x != &L->nil && x->chave != k) x = x->prox;
    return x == &L->nil ? NULL : x;
}
```

PRÁTICA EM C

Lista com sentinela em C (3/4)

```
0
void remove_no(No *x) {                          // LIST-DELETE': O(1) dado o nó
    x->ant->prox = x->prox;
    x->prox->ant = x->ant;
    free(x);
}

int main(void) {
    Lista L;
    inicia(&L);
    char cmd[4];
    int k;
    while (scanf("%3s", cmd) == 1) {
        if (cmd[0] == 'i' && scanf("%d", &k) == 1) insere(&L, k);
        else if (cmd[0] == 'b' && scanf("%d", &k) == 1)
```

PRÁTICA EM C

Lista com sentinela em C (4/4)

```
0
            printf(busca(&L, k) ? "achou %d\n" : "não achou %d\n", k);
        else if (cmd[0] == 'r' && scanf("%d", &k) == 1) {
            No *x = busca(&L, k);
            if (x) remove_no(x);
        } else if (cmd[0] == 'p') {
            printf("lista:");
            for (No *x = L.nil.prox; x != &L.nil; x = x->prox) printf(" %d", x->chave);
            printf("\n");
        }
    }
    while (L.nil.prox != &L.nil) remove_no(L.nil.prox);
    return 0;
}
```



Experimento

- Inserir n vezes no início: vetor desloca $n(n-1)/2$; lista, nada
- Percorrer: tempo por elemento de vetor e de lista, n até 2^{24}
- Nós da lista em ordem aleatória na memória

EXPERIMENTO

Ponteiros contra vizinhança

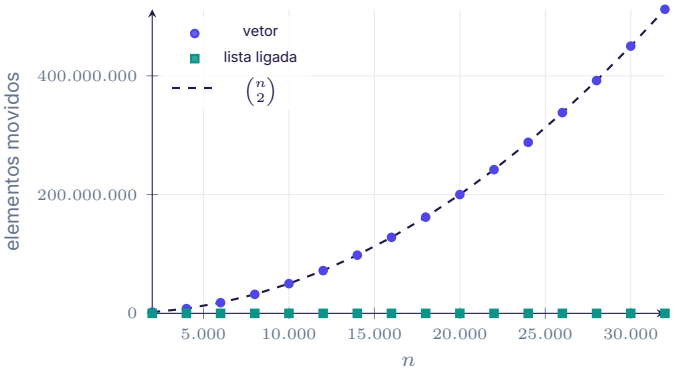
Hipótese. Inserir no início de um vetor desloca todos os elementos: n inserções movem $n(n-1)/2$ elementos; a lista só ajusta ponteiros. Para percorrer, os dois são $\Theta(n)$ no modelo RAM. Mas a CPU busca a memória em blocos (linhas de cache): o vetor aproveita cada bloco, a lista espalhada não. Esperamos o mesmo $\Theta(n)$ com **constantes muito diferentes**.

Método

1. Contagem exata dos deslocamentos
2. Percurso: $n = 2^{10}$ a 2^{24}; cerca de 33 milhões de visitas por medida
3. Nós da lista embaralhados (Fisher–Yates)
4. Tempo dividido pelo número de visitas

EXPERIMENTO

Inserir no início



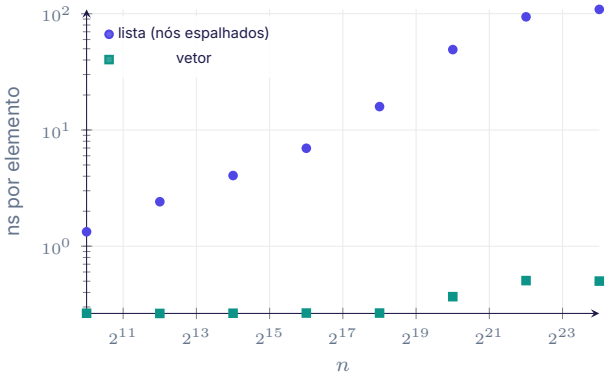
n	medido	medido ÷ $\binom{n}{2}$
2.000	1.999.000	1,000
8.000	31.996.000	1,000
14.000	97.993.000	1,000
20.000	199.990.000	1,000
26.000	337.987.000	1,000
32.000	511.984.000	1,000

✓ desvio máx. 0,0%

O vetor move exatamente $n(n-1)/2$ elementos; a lista, zero.

EXPERIMENTO

Percorrer: o cache decide



Enquanto tudo cabe no cache, a lista custa 1 a 4 ns por nó. Fora do cache (milhões de nós), passa de 100 ns: cada nó é uma ida à RAM.

EXPERIMENTO

O que o experimento mostrou

Inserir no início confirma a teoria: $n(n-1)/2$ movimentos no vetor, nenhum na lista. No percurso, os dois são $\Theta(n)$, mas a lista espalhada fica até **200 vezes** mais lenta que o vetor quando sai do cache. **O modelo RAM não vê a hierarquia de memória**; a medição vê.

Reproduza: `gcc -O2 aula17_experimento.c -o exp -lm && ./exp`



Conexões

- Sistemas operacionais
- Alocação de memória
- Arquitetura de computadores
- Editores de texto

CONEXÕES

Onde isso aparece na computação

Sistemas operacionais

O kernel Linux usa listas duplamente ligadas circulares com sentinela (`list_head`) em processos, arquivos abertos e filas de espera.

Alocação de memória

`malloc` guarda os blocos livres em listas livres, como na Figura 10.5; coletores de lixo percorrem listas de objetos.

Arquitetura de computadores

O experimento mostra a hierarquia de memória: L1 (≈1 ns), L2, L3 e RAM (≈100 ns). Estruturas *cache-friendly* vencem na prática.

Editores de texto

Editores mantêm o texto em listas de linhas ou em *piece tables* ligadas: inserir no meio de um documento grande custa $O(1)$.

PARA CASA

Para casa

Exercícios

1. (10.2-1) É possível inserir em $O(1)$ numa lista simplesmente ligada? E remover?
2. (10.2-2) Implemente uma pilha com lista simplesmente ligada.
3. (10.2-7) Inverta uma lista simplesmente ligada em $\Theta(n)$, sem memória extra.
4. (10.3-2) Implemente ALLOCATE-OBJECT e FREE-OBJECT para a representação em um só vetor.
5. No experimento, aloque os nós em ordem (sem embaralhar). O que acontece com o tempo da lista? Por quê?

Leitura
Cormen et al., seções 10.2 e 10.3.

Código
josedihego.net/estruturas